

PERANCANGAN DAN IMPLEMENTASI LAYANAN
PENILAIAN KODE PROGRAM BERBASIS
LARGE LANGUAGE MODEL

Disusun sebagai syarat kelulusan mata kuliah
IF4091/Penulisan Proposal

Oleh
Valentino Chrystie Triadi
NIM : 13522164



PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO & INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
Oktober 2025

PERANCANGAN DAN IMPLEMENTASI LAYANAN
PENILAIAN KODE PROGRAM BERBASIS
LARGE LANGUAGE MODEL

Laporan Proposal Tugas Akhir

Oleh

Valentino Chrylie Triadi

NIM : 13522164

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Bandung, 23 Desember 2025

Mengetahui,

Pembimbing,

Dr. techn. Saiful Akbar, S.T, M.T.

NIP. 197405091998031002

DAFTAR ISI

BAB I PENDAHULUAN.....	6
I.1 Latar Belakang.....	6
I.2 Rumusan Masalah.....	7
I.3 Tujuan dan Ukuran Keberhasilan Pencapaian.....	8
I.4 Batasan Masalah.....	9
I.5 Metodologi.....	9
BAB II KAJIAN PUSTAKA.....	11
II.1 Penilaian Kode Program.....	11
II.1.1 Black-Box.....	12
II.1.2 White-Box.....	13
II.2 Large Language Model.....	14
II.2.1 Arsitektur.....	14
II.2.1.1 Autoregressive Language Models.....	15
II.2.1.2 Masked Language Models.....	15
II.2.1.3 Encoder-Decoder Models.....	16
II.2.2 Cara Kerja.....	16
II.2.2.1 Pelatihan (training).....	16
II.2.2.2 Bobot (weights).....	17
II.2.2.3 Inferensi (inference).....	18
II.3 Adaptasi Large Language Model.....	19
II.3.1 Pre-training.....	19
II.3.2 Fine Tuning.....	19
II.3.3 Prompting.....	20
II.3.3.1 Zero-Shot dan Few-Shot.....	21
II.3.3.2 Single-Turn Instruction dan Multi-Turn Instruction.....	21
II.3.3.3 Chain Of Thought (CoT).....	21
II.4 Penelitian Terkait.....	22
BAB III ANALISIS MASALAH DAN RANCANGAN SOLUSI.....	26
III.1 Analisis Masalah.....	26
III.2 Gambaran Umum Solusi.....	27
III.2.1 Strategi Prompt Engineering.....	27
III.2.2 Arsitektur Layanan.....	28

BAB IV RENCANA PELAKSANAAN.....	29
IV.1 Jadwal.....	29
IV.1.1 Design.....	30
IV.1.2 Implementation.....	30
IV.1.3 Experiment.....	30
IV.1.4 Testing.....	31
IV.1.5 Report.....	31
IV.2 Risiko.....	31

DAFTAR LAMPIRAN

Lampiran A. Analisis Solusi.....	36
Lampiran B. Gambaran Solusi.....	40

DAFTAR GAMBAR

Gambar II.1 Pengujian black box.....	13
Gambar IV.1 Jadwal Pelaksanaan Tugas Akhir.....	29
Gambar B.1 Gambaran Umum Solusi.....	40
Gambar B.2 Gambar Arsitektur Layanan Penilaian.....	40

DAFTAR TABEL

Tabel II.1 Tabel rubrik kriteria penilaian (Raj dkk., 2024).....	23
--	----

BAB I

PENDAHULUAN

Bab ini menyajikan fondasi dan konteks menyeluruh mengenai pelaksanaan tugas akhir. Bab ini berfungsi sebagai peta jalan yang menguraikan seluruh rencana penelitian. Bab ini berisi latar belakang topik tugas akhir, masalah yang akan diselesaikan, tujuan dan ukuran keberhasilan tugas akhir, batasan masalah, dan metodologi penyelesaian tugas akhir. Dengan demikian, pembaca diharapkan dapat memperoleh gambaran komprehensif terkait dasar pemikiran, arah, dan kerangka kerja dari seluruh proses penelitian yang disajikan.

I.1 Latar Belakang

Penilaian kode program merupakan komponen esensial dalam pendidikan ilmu komputer. Secara tradisional, proses ini sering dilakukan secara manual oleh dosen atau asisten pengajar. Pendekatan manual memungkinkan pemberian umpan balik yang personal dan mendalam. Namun, metode ini memiliki kelemahan signifikan, di antaranya adalah membutuhkan waktu dan tenaga yang cukup tinggi, potensi bias dan subjektivitas, serta inkonsistensi dalam penilaian.

Untuk mengatasi tantangan tersebut, dikembangkan sistem penilaian otomatis seperti *Online Judge*. *Online Judge* menawarkan keunggulan dalam hal objektivitas, kecepatan, dan skalabilitas. Akan tetapi, *Online Judge* memiliki keterbatasan yaitu umpan balik yang diberikan seringkali terbatas pada status "lulus/gagal" (menggunakan teknik *black-box*) dan kurang memperhatikan aspek kualitas kode yang penting, seperti keterbacaan (*readability*), *maintainability*, atau efisiensi algoritma.

Dalam beberapa tahun terakhir, *Large Language Model* (LLM) seperti GPT telah menunjukkan kemampuan luar biasa dalam memproses struktur linguistik yang kompleks dan menghasilkan teks yang menyerupai manusia. Kemampuan ini,

didukung oleh pelatihan ekstensif pada contoh kode pemrograman. Hal ini membuka potensi baru dalam penilaian otomatis. LLM menawarkan mekanisme baru untuk mengatasi tantangan dalam menghasilkan umpan balik yang personal dan mendalam. Model-model ini dapat memberikan penjelasan kesalahan, menawarkan petunjuk *debugging*, dan menyarankan perbaikan kode, jauh melampaui umpan balik standar yang diberikan *Online Judge*.

Penelitian terkait telah menunjukkan bahwa LLM dapat digunakan untuk evaluasi kode yang komprehensif berdasarkan berbagai dimensi, tidak hanya fungsionalitas, tetapi juga kualitas kode dan efisiensi algoritma. Namun, penelitian tersebut masih belum diimplementasikan ke dunia pendidikan secara langsung dan masih belum ada penelitian yang mengatasi penilaian dengan jumlah mahasiswa yang cukup besar secara konsisten dan memiliki kesesuaian yang tinggi. Oleh karena itu, terdapat kebutuhan untuk merancang dan mengimplementasikan sebuah layanan yang memanfaatkan kemampuan LLM untuk menyediakan penilaian kode program yang tidak hanya otomatis dan objektif, tetapi juga komprehensif dan kaya akan umpan balik kualitatif. Layanan tersebut juga harus memiliki konsistensi yang tinggi dalam setiap hasil yang diberikannya.

I.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Diperlukan layanan yang mampu melakukan penilaian kode program secara otomatis berdasarkan permasalahan yang diberikan.
2. Hasil penilaian dan umpan balik yang diberikan oleh layanan harus memiliki konsistensi yang tinggi dan kesesuaian dengan metode penilaian manual.

Oleh karena itu, perlu adanya sebuah layanan penilaian kode program berbasis *Large Language Model* yang mampu mengotomatisasi penilaian tidak hanya pada aspek kebenaran fungsional (*black-box*) tetapi juga pada aspek kualitas kode.

I.3 Tujuan dan Ukuran Keberhasilan Pencapaian

Tujuan utama yang akan dicapai dalam pelaksanaan tugas akhir ini adalah:

1. Merancang arsitektur layanan dan mengimplementasikan layanan tersebut sehingga mampu menerima kiriman kode program dan menghasilkan penilaian (skor) serta umpan balik kualitatif berdasarkan rubrik yang telah ditentukan.
2. Mengevaluasi dan menganalisis kinerja layanan dalam memberikan penilaian, khususnya dalam hal akurasi, konsistensi, dan kualitas umpan balik yang dihasilkan dibandingkan dengan penilaian manual oleh manusia.

Ukuran keberhasilan pencapaian dari tujuan tersebut adalah:

1. Berhasil diimplementasikannya prototipe layanan penilaian kode program berbasis LLM.
2. Hasil penilaian (skor) yang diberikan oleh layanan LLM menunjukkan korelasi positif yang kuat (misalnya, $\text{PearsonR} \approx 0.81$) dengan skor yang diberikan oleh penilai manusia.
3. Layanan mampu menghasilkan umpan balik secara konsisten (memiliki *repeatability* yang tinggi) dan terstruktur sesuai rubrik pada berbagai kriteria (fungsionalitas, efisiensi, kualitas kode).
4. Layanan mampu mendapatkan umpan balik positif dari pengguna (dosen atau penilai) berupa kesesuaian penilaian yang diberikan oleh layanan dengan pengguna.

I.4 Batasan Masalah

Batasan-batasan masalah (ruang lingkup) dalam pelaksanaan tugas akhir ini adalah sebagai berikut:

1. Penelitian ini berfokus pada perancangan dan implementasi layanan, bukan pada *pre-training* atau *full fine-tuning* model LLM dari awal.
2. Model LLM yang digunakan akan terbatas pada model yang tersedia melalui API (misalnya GPT-3.5, GPT-4 , Gemini 2.5 Flash) atau model *open-source* yang relevan (misalnya Zephyr-7B).
3. Bahasa pemrograman yang didukung oleh layanan akan dibatasi pada beberapa bahasa pemrograman yang pengetahuannya telah dimiliki oleh model seperti Python, Java, C, C#, C++, Golang, PHP, Javascript, Ruby, Rust, Swift, Dart, Kotlin, Haskell, R, dan lain-lain. Adapun beberapa bahasa lainnya dapat ditambahkan apabila ada pengetahuan tambahan yang diberikan misalnya notasi algoritmik.

I.5 Metodologi

Tahapan yang akan dilalui selama pelaksanaan tugas akhir ini adalah sebagai berikut:

1. Analisis Kebutuhan dan Perancangan Sistem

Menganalisis kebutuhan fungsional dan non-fungsional layanan. Merancang arsitektur sistem secara keseluruhan, termasuk alur data, antarmuka, dan integrasi API LLM.

2. Perancangan Rubrik dan *Prompt Engineering*

Mengembangkan rubrik penilaian terstruktur yang detail, mengacu pada penelitian sebelumnya (seperti rubrik 22 kriteria oleh Raj dkk.). Merancang dan menguji *prompt* (instruksi) untuk LLM, termasuk

menerapkan beberapa strategi *prompting* seperti *Chain of Thought* (CoT) agar LLM dapat "berpikir" langkah demi langkah sesuai rubrik.

3. Implementasi Prototipe Layanan

Melakukan implementasi layanan berdasarkan arsitektur yang telah dirancang, mengintegrasikan semua komponen menjadi satu sistem yang fungsional.

4. Pengujian dan Evaluasi

Mengumpulkan dataset kode program (misalnya tugas mahasiswa). Melakukan evaluasi dengan membandingkan hasil (skor dan umpan balik) dari layanan LLM dengan hasil penilaian dari penilai manusia (ahli). Kinerja akan diukur menggunakan metrik korelasi (misal: Pearson R) dan konsistensi (misal: ICC).

5. Analisis Hasil dan Penulisan Laporan

Menganalisis data hasil evaluasi untuk mengukur efektivitas, akurasi, dan potensi bias (misalnya "halusinasi") dari layanan. Menarik kesimpulan dan menyusun laporan tugas akhir.

BAB II

KAJIAN PUSTAKA

Bab ini menyajikan tinjauan sistematis terhadap literatur dan penelitian terdahulu yang relevan dengan topik tugas akhir. Kajian pustaka ini berfungsi sebagai fondasi teoritis dan kontekstual untuk mengidentifikasi celah penelitian (research gap) dan mengembangkan kerangka kerja yang akan digunakan dalam penelitian ini. Garis besar bahasan studi literatur akan berupa penilaian kode program, *large language model*, adaptasi *large language model*, dan penelitian terkait. Hasil dari studi literatur diharapkan dapat memberi pemahaman yang komprehensif terkait topik tugas akhir.

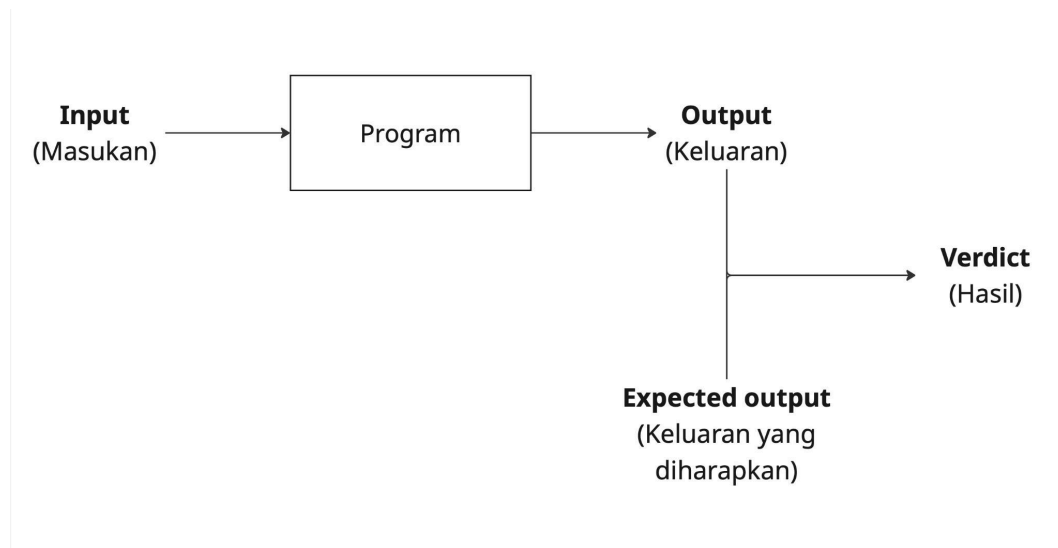
II.1 Penilaian Kode Program

Penilaian kode program secara tradisional masih sering dilakukan secara manual. Penilaian kode program secara manual umumnya dilakukan oleh dosen atau asisten pengajar dengan menilai aspek seperti kebenaran, efisiensi, keterbacaan, dokumentasi, dan kualitas kode secara keseluruhan. Penilaian ini seringkali melibatkan pemberian skor, komentar, atau umpan balik personal. Kelebihan dari penilaian secara manual adalah dapat memberikan umpan balik yang personal dan mendalam, menilai solusi kreatif atau kode yang tidak sesuai dengan standar namun merupakan alternatif jawaban, dan dapat memaklumi kesalahan penulisan yang seharusnya tidak menjadi salah satu aspek penilaian terbesar. Namun, terdapat beberapa kekurangan dan tantangan dalam melakukan penilaian secara manual, yaitu konsistensi nilai yang diberikan tidak terjamin, waktu dan tenaga yang dibutuhkan cukup besar, bias dan subjektif dalam penilaian, kualitas umpan balik yang bisa menurun apabila permasalahan yang akan diselesaikan cukup kompleks. Menanggapi beberapa tantangan tersebut, tim penilai menggunakan rubrik penilaian sebagai acuan dalam melakukan penilaian secara manual.

Dalam meningkatkan konsistensi dan efisiensi dari penilaian kode program, dibuat sistem untuk mengotomatisasi penilaian seperti *Online Judge*. *Online Judge* telah menjadi standar dalam penilaian otomatis kode pemrograman. *Online Judge* memiliki beberapa kelebihan seperti penilaian yang lebih objektif, cepat, dan efektif. Kelebihan *Online Judge* meliputi kecepatan, objektivitas, dan skalabilitas, namun umpan balik yang diberikan seringkali terbatas pada hasil lulus/gagal dan kurang memperhatikan aspek kualitas kode seperti maintainability dan readability. *Online Judge* juga belum dapat melakukan penilaian ke aspek yang lebih luas seperti kreativitas dari penyelesaian masalah, seberapa efisien kode program yang dibangun, dan aspek-aspek lainnya. *Online Judge* umumnya menggunakan teknik black-box dan white-box untuk melakukan penilaian.

II.1.1 Black-Box

Pengujian dengan teknik black box adalah pengujian yang dilakukan berdasarkan kasus uji yang didesain dari spesifikasi program. Pengujian black box memiliki benefit dengan tidak ketergantungan dengan informasi kode program (Zhichao dkk., 2022). Black box sendiri dilakukan dengan mencocokkan kasus uji yang diberikan dengan hasil keluaran dari menjalankan program. Hasil yang didapat dari pengujian black box adalah kesesuaian antara hasil keluaran program dengan ekspektasi keluaran yang diberikan dalam kasus uji. Berikut gambaran alur pengujian dengan black box.



Gambar II.1 Pengujian black box

Pengujian dengan black box merupakan pengujian yang sangat ketat. Pengujian ini memiliki beberapa kelebihan seperti kemampuan untuk menyesuaikan dengan spesifikasi permasalahan, dapat mengurangi bias dari penguji, tidak memerlukan kemampuan programming untuk melakukan pengujian, dan efisien untuk pengujian berskala besar. Dengan kelebihan tersebut, pengujian black box ini juga memiliki beberapa kekurangan, seperti adanya kemungkinan terjadi pengujian yang redundan, pembuatan kasus uji yang cukup sulit apabila permasalahan kurang jelas, kurang efisien apabila digunakan untuk melakukan pengujian sistem yang kompleks, dan hasil dari pengujian ini seringkali dianggap sangat bagus padahal belum tentu keseluruhan aspek teruji.

II.1.2 White-Box

Pengujian dengan teknik white box adalah pengujian yang dilakukan dengan memperhatikan struktur dari sebuah aplikasi atau kode (Akanksha dkk., 2017). Pengujian white box dapat berbeda untuk konteks pengujian yang berbeda. Dalam konteks pemrograman, pengujian white box dapat melakukan pengujian terhadap beberapa aspek seperti, logika, struktur kode, dan tingkat efisien atau kompleksitas dari kode. Pengujian white box memerlukan penguji yang memahami terkait konteks program yang akan diuji. Hasil dari pengujian ini dapat

berupa umpan balik atau penilaian terhadap struktur kode program. Berikut gambaran alur pengujian dengan white box.

Pengujian dengan white box merupakan pengujian yang luas. Pengujian ini memiliki beberapa kelebihan seperti kemampuan untuk melakukan pengujian yang lebih menyeluruh dan pengujian ini dapat membantu untuk melakukan optimasi kode. Dengan kelebihan tersebut, pengujian white box juga memiliki beberapa kekurangan, seperti kurang tersedianya kakas untuk seluruh konteks pemrograman, kesulitan dalam mengelola kasus uji apabila terjadi perubahan kebutuhan program, dan adanya kemungkinan beberapa kasus yang tidak teruji.

II.2 *Large Language Model*

Large Language Model (LLM) merupakan model bahasa berskala besar berbasis arsitektur Transformer yang dilatih pada data teks masif untuk memahami dan menghasilkan bahasa alami. LLM dilatih untuk belajar memprediksi kata berikutnya berdasarkan kata-kata sebelumnya. Tujuannya adalah agar model dapat mempelajari representasi fitur laten yang generik dari sejumlah besar teks yang tersedia secara alami, sehingga memungkinkan untuk memahami dan menghasilkan bahasa alami. Beberapa contoh LLM adalah BERT dan GPT.

II.2.1 *Arsitektur*

Hampir semua LLM populer saat ini didasarkan pada arsitektur transformer. Inti dari arsitektur transformer adalah adanya mekanisme *multi-head self-attention*, yang memungkinkan model menangkap dependensi jarak jauh secara efisien. Mekanisme ini memperkenalkan setiap kata untuk "memperhatikan" atau menangkap konteks dari semua kata lain dalam sebuah urutan saat memprosesnya (Vaswani dkk., 2017). LLM modern saat ini terbagi menjadi tiga kelas, yaitu *autoregressive language models*, *masked language models*, dan *encoder-decoder models*.

II.2.1.1 Autoregressive Language Models

Model ini dilatih untuk memprediksi kata berikutnya (x_i) dengan pengetahuan dari kata-kata sebelumnya ($x_1, x_2, \dots, x_{i-1}$). Tujuan utama dari proses pelatihan model ini adalah untuk memaksimalkan *log-likelihood*.

$$\sum_i \log(P(x_i | x_1, x_2, \dots, x_{i-1}); \theta_T)$$

Dengan θ_T adalah parameter dari model. Model ini hanya menggunakan bagian autoregressive decoder dari arsitektur transformer. Model ini menggunakan beberapa lapisan decoder dengan *masked self-attention*, hal ini memungkinkan model untuk memperhatikan semua token sebelumnya saat memprediksi token berikutnya.

II.2.1.2 Masked Language Models

Model ini menggunakan bagian *bidirectional encoder* dari transformer. Model ini dilatih untuk memprediksi kata-kata yang “ditutupi” (masked) berdasarkan konteks di sebelah kiri dan kanannya (dua arah). Selama pelatihan, beberapa kata dipilih secara acak untuk ditutupi, baik menggunakan token khusus [MASK] atau diganti dengan token acak. Proses ini memaksa model untuk mengumpulkan informasi secara bidirectional (dua arah) untuk memprediksi token asli di posisi yang ditutupi.

$$\sum_i \log(P(x_i | x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_n); \theta_T)$$

Dengan $m_i \in \{0, 1\}$ menandakan apakah x_i sedang “ditutupi” atau tidak dan θ_T adalah parameter dari transformer encoder. Model ini menggunakan beberapa lapisan encoder dan menggunakan masked self-attention untuk memperhatikan token lain di kedua arah secara bersamaan. Sifat non-autoregresif ini memungkinkan komputasi yang diparalelkan, sehingga model ini seringkali lebih efisien pada saat inferensi dan juga efisien selama pelatihan. Contoh populer dari model MLM ini adalah BERT, RoBERTa, dan XLM-R.

II.2.1.3 Encoder-Decoder Models

Model ini menggunakan arsitektur transformer secara penuh, yang terdiri dari *bidirectional encoder* dan *autoregressive decoder*. Model ini dilatih untuk merekonstruksi urutan teks asli dari urutan teks yang "dirusak" (corrupted). Model ini belajar untuk menghasilkan atau memprediksi sekuens token $y_1, \dots, y_n$ dari input $x_1, \dots, x_m$. Tujuan utama dari proses pelatihan model ini adalah untuk memaksimalkan *log-likelihood*.

$$\sum_i \log(P(y_1, \dots, y_n \mid x_1, \dots, x_m); \theta_T)$$

Dengan θ_T adalah parameter dari transformer encoder-decoder penuh. Model ini dilatih dengan menggunakan beberapa masukan berupa urutan token yang telah dimodifikasi (dirusak) dengan tujuan untuk merekonstruksi urutan aslinya sebagai keluaran. Bentuk-bentuk kerusakan ini bisa berupa rotasi dokumen, permutasi kalimat, pengisian teks (*text infilling*), atau penghapusan/penutupan token. Karena sifatnya yang *sequence-to-sequence* (seq2seq), model *encoder-decoder* mudah di-*fine-tune* (disempurnakan).

II.2.2 Cara Kerja

Cara kerja LLM dapat dibedakan menjadi tiga proses berbeda yang saling berkorelasi satu sama lain. Tiga proses tersebut adalah pelatihan (*training*), strategi penggunaan bobot (*weights*), dan penggunaan untuk inferensi (*inference*).

II.2.2.1 Pelatihan (*training*)

Pelatihan LLM pada umumnya mengikuti paradigma “Pre-Train then Fine-Tune”. Pada tahap *pre-training*, model dilatih dengan korpus bahasa yang sangat besar agar model mampu melakukan pemodelan bahasa seperti memprediksi kata berikutnya atau *masked-words*. Pelatihan pada tahap *pre-training* ini bersifat *self-supervised* dimana model dilatih dengan menggunakan korpus data teks yang tidak berlabel. Tujuan dari pelatihan *pre-training* ini adalah agar model dapat mempelajari representasi bahasa yang fundamental. Setelah tahap *pre-training*

selesai dilakukan, LLM sudah dapat digunakan untuk keperluan merepresentasikan bahasa secara mendasar. Namun, ada beberapa kasus dimana LLM memerlukan pengetahuan lebih terhadap sebuah domain tertentu. Sehingga, diperlukan adaptasi dari LLM yang fundamental menjadi LLM yang mampu memahami konteks lebih sesuai dengan domain tertentu. *Fine-tuning* dilakukan untuk mengadaptasi LLM fundamental ke domain tertentu. Pada tahap *fine-tuning*, model dilatih kembali dengan menggunakan dataset berlabel yang lebih kecil dan spesifik ke domain tertentu.

II.2.2.2 Bobot (*weights*)

Salah satu parameter dalam proses pelatihan LLM adalah bobot atau *weights*. Bobot sendiri merupakan parameter internal yang dipelajari selama proses pelatihan. Hasil dari *pre-training* LLM sendiri merupakan bobot-bobot tertentu yang mampu merepresentasikan bahasa. Pada tahap *fine-tuning*, bobot-bobot yang telah ada dari proses *pre-training* disesuaikan kembali agar dapat merepresentasikan konteks dalam domain tertentu. Terdapat beberapa strategi dalam penyesuaian bobot pada tahap *fine-tuning*, yaitu *contextual embeddings*, *full fine-tuning*, dan *efficient fine-tuning*.

Contextual embeddings adalah salah satu strategi yang paling simple. Strategi ini membekukan model dan tidak melakukan perubahan bobot sama sekali. Keluaran dari model yang dibekukan tersebut yang berupa *words embeddings* akan digunakan sebagai masukan dari arsitektur terpisah yang dilatih untuk konteks spesifik tertentu. Strategi *contextual embeddings* dapat digunakan dalam beberapa skenario, seperti keterbatasan data atau daya komputasi, tugas yang sangat kompleks, dan *unsupervised-task*.

Full fine-tuning adalah strategi yang melibatkan penyesuaian bobot pada sebagian atau setiap lapisan model dan menambahkan satu atau dua lapisan keluaran sederhana. Lapisan keluaran dan model dilatih bersama secara *end-to-end*. Namun, terdapat permasalahan terkait *learning rate* yang harus digunakan dalam pelatihan. Umumnya, model yang sudah dilatih tidak memerlukan *learning rate*

yang besar sedangkan lapisan keluaran yang baru dibentuk memerlukan *learning rate* yang besar. Solusinya adalah melakukan pembekuan sementara beberapa lapisan model terlebih dahulu dan membuka lapisan demi lapisan secara bertahap seiring dengan proses pelatihan lapisan keluaran berlangsung. Solusi lain adalah dengan melakukan penyesuaian beberapa lapisan teratas saja. Sebagai contoh, lapisan bawah pada BERT mengandung informasi sintaktik sedangkan lapisan atas mengandung informasi yang lebih kompleks seperti semantik dan koreferensi.

Efficient fine-tuning adalah strategi yang bertujuan untuk menyempurnakan hanya sejumlah kecil bobot model. Strategi ini dapat dibedakan menjadi dua tipe, yaitu melakukan penyesuaian jaringan kecil terpisah yang ditambahkan ke model *pre-trained* sementara model tersebut masih beku dan memilih hanya sejumlah kecil bobot internal model *pre-trained* untuk disempurnakan atau disimpan. Contoh utama dari tipe pertama adalah *Adapter modules*, yang menambahkan sekumpulan kecil bobot baru di setiap lapisan transformer. Selama pelatihan, seluruh bobot model dibekukan dan hanya bobot adapter yang disempurnakan. Hal tersebut sangat efisien, memungkinkan berbagi bobot, dan mencegah *catastrophic forgetting*. Metode serupa lainnya termasuk *side-tuning* (melatih jaringan "samping" yang ringan) dan *diff-pruning* (menambahkan vektor perbedaan yang *sparse* ke parameter beku). Jenis kedua, yang memodifikasi bobot model itu sendiri, mencakup BitFit (membatasi *fine-tuning* hanya pada *bias terms* atau sekitar 0,1% parameter) dan masking (membekukan semua bobot model namun melatih *mask* biner yang *sparse* untuk setiap tugas guna memilih bobot mana yang relevan).

II.2.2.3 Inferensi (*inference*)

Inferensi adalah proses menggunakan model yang sudah dilatih untuk membuat prediksi. Proses inferensi diawali dengan model yang sudah dilatih menerima *prompt* dan mengubahnya menjadi token. Model kemudian memproses token-token ini dalam satu *forward pass* untuk memprediksi probabilitas setiap kata yang mungkin muncul berikutnya. Kata yang baru dipilih kemudian

ditambahkan ke akhir *prompt* asli. Seluruh urutan tersebut kemudian dimasukkan kembali ke model untuk memprediksi kata berikutnya. Proses ini terus berulang hingga model memutuskan untuk berhenti dengan menghasilkan token khusus "stop" atau telah mencapai batas panjang maksimum.

II.3 Adaptasi *Large Language Model*

LLM adalah model bahasa yang mampu merepresentasikan bahasa natural yang digunakan oleh manusia dalam berkomunikasi. Kemampuan LLM sangat beragam, dimulai dari yang fundamental hingga spesifik ke domain tertentu. Kemampuan LLM tersebut didapat dari fase adaptasi model terhadap data yang diberi.

II.3.1 *Pre-training*

Pre-training adalah tahap dimana model diadaptasi untuk melakukan tugas-tugas fundamental. Pada tahapan *pre-training*, model diberikan data besar untuk belajar memodelkan bahasa sehingga dapat memberikan hasil berupa representasi dari bahasa yang biasa digunakan manusia untuk berkomunikasi. Hasil adaptasi dari tahap *pre-training* adalah model yang mampu memprediksi kata demi kata untuk menyusun kalimat yang dapat dipahami oleh manusia. Tahap ini dibatasi dengan kemampuan model untuk melakukan tugas-tugas fundamental saja, bukan tugas yang spesifik ke domain tertentu.

II.3.2 *Fine Tuning*

Fine-tuning adalah tahap dimana model yang sudah dilatih pada tahap *pre-training* dilatih kembali untuk beradaptasi dengan tugas-tugas yang lebih spesifik. Hasil dari tahap ini adalah model yang dapat melakukan tugas-tugas yang lebih spesifik berdasarkan latihan yang dilakukan. *Fine-tuning* sendiri dapat dilakukan dengan beberapa strategi atau pendekatan, yaitu *transfer learning*, *instruction tuning*, *alignment tuning*, *adapter tuning*, dan *prompt tuning*. ***Transfer learning*** adalah proses pelatihan model dengan data yang spesifik ke task tertentu dengan harapan model dapat belajar hal-hal baru dari data yang lebih spesifik.

Instruction Tuning adalah *fine-tuning* model dengan menggunakan data berformat instruksi. Data ini berupa sebuah instruksi beserta pasangan masukan dan keluarannya. Instruksi tersebut umumnya mencakup data *multi-task* dalam bahasa alami sederhana yang berfungsi sebagai panduan model dalam memberikan respon. Pendekatan ini dapat meningkatkan kemampuan generalisasi *zero-shot* model dan kinerjanya pada tugas-tugas spesifik. **Alignment Tuning** adalah salah satu pendekatan *fine-tuning* yang menggunakan umpan balik dari manusia. Pendekatan ini memiliki tujuan untuk membuat model menjadi *helpful*, *honest*, dan *harmless* atau biasa disebut dengan “HHH”. Proses adaptasi pada pendekatan ini adalah dengan melakukan *alignment* terhadap parameter model dan memastikan model berjalan sesuai dengan keinginan dan nilai-nilai manusia. **Adapter Tuning** adalah pendekatan *fine-tuning* dengan menambahkan beberapa *trainable parameters* pada blok transformer. Lapisan adapter ini biasanya terdiri dari urutan *downscaling* fitur, non-linearitas, dan *upscaling*. **Prompt Tuning** adalah pendekatan *fine-tuning* dengan melakukan penyesuaian terhadap sejumlah sedikit parameter tambahan (0.001%-3%). Kemudian, pendekatan ini menggabungkan parameter yang dilatih tadi dengan *model embeddings*. Pendekatan ini awalnya menggabungkan *prompt* yang spesifik dengan *input embeddings*. Namun karena *discrete prompt* menimbulkan ketidakstabilan, maka *prompts* akan di-*encode* melalui *learnable mapping* dalam P-Tuning yang biasa disebut dengan *continuous prompts*. Hasilnya, hanya *prompt encoder* yang dapat dilatih dalam model tersebut.

II.3.3 Prompting

Prompting adalah metode untuk melakukan query terhadap LLM untuk me-*generate* respon. *Prompting* dilakukan untuk menyesuaikan respon LLM terhadap konteks yang diinginkan dari masukan query tanpa perlu melakukan *fine-tuning*. Terdapat beberapa metode *prompting* sebagai berikut.

II.3.3.1 *Zero-Shot* dan *Few-Shot*

Zero-shot adalah metode *prompting* paling dasar dan paling umum digunakan. Metode ini hanya menggunakan pengetahuan yang dimiliki oleh LLM tanpa adanya informasi tambahan berupa contoh dari query yang diharapkan. Sedangkan *Few-shot* atau *In-context Learning* adalah metode *prompting* yang memberikan contoh masukan keluaran dalam *prompt*. Metode ini membuat model beradaptasi terhadap contoh masukan dan keluaran yang diberikan dengan tujuan model dapat menerima masukan dan memberikan keluaran yang kurang lebih sama seperti contoh.

II.3.3.2 *Single-Turn Instruction* dan *Multi-Turn Instruction*

Single-Turn Instruction adalah metode *prompting* dengan menggunakan hanya satu query. Query tersebut akan berisi seluruh informasi relevan yang dibutuhkan oleh LLM. Sedangkan *Multi-Turn Instruction* adalah metode *prompting* dimana dilakukan beberapa kali proses query. Query awal akan berisi seluruh informasi relevan yang dibutuhkan LLM. Kemudian, setelah LLM berhasil memberikan respon, diberikan query tambahan untuk memberikan umpan balik terhadap hasil keluaran model LLM tersebut. Proses ini biasanya akan dilakukan berulang kali hingga respon yang diberikan dari LLM sesuai dengan apa yang diharapkan.

II.3.3.3 *Chain Of Thought (CoT)*

Chain Of Thought (CoT) adalah metode *prompting* yang membuat model menghasilkan alasan atau langkah berpikir terlebih dahulu sebelum mengeluarkan jawaban akhir. Dengan adanya alasan atau langkah berpikir tersebut, LLM dapat menunjukkan kemampuan penalaran khususnya dalam menyelesaikan tugas-tugas yang cukup tinggi tingkat kesulitannya. Kojime dkk. (2022) menemukan bahwa LLM dapat melakukan CoT dalam mode *zero-shot* hanya dengan menambahkan frasa “Let’s think step by step”.

II.4 Penelitian Terkait

Penelitian terkait mengenai penilaian tugas pemrograman menunjukkan bahwa integrasi LLM menawarkan mekanisme baru yang menjanjikan untuk mengatasi tantangan dalam menghasilkan umpan balik yang personal dan mendalam. Secara tradisional, penilaian tugas pemrograman merupakan proses yang memakan waktu, terutama dalam konteks kelas besar atau Massive Open Online Courses (MOOCs). Meskipun telah banyak dikembangkan alat penilaian otomatis (automated assessment tools) dalam pendidikan ilmu komputer, alat-alat ini seringkali terbatas pada pemeriksaan kebenaran fungsional dan gagal dalam memberikan saran yang membantu atau menilai kualitas kode secara komprehensif. LLM seperti GPT (Generative Pre-trained Transformer) dari OpenAI (GPT-3.5, GPT-4), telah menarik perhatian karena kemampuannya memproses struktur linguistik yang kompleks dan menghasilkan teks menyerupai manusia. Kemampuan ini, didukung oleh pelatihan ekstensif pada contoh kode pemrograman, memungkinkan LLM untuk memberikan umpan balik yang secara otomatis menjelaskan kesalahan, menawarkan petunjuk *debugging*, dan menyarankan perbaikan kode, melampaui umpan balik standar (Pankiewicz & Baker, 2023).

Perancangan layanan penilaian berbasis LLM berfokus pada evaluasi kode yang komprehensif berdasarkan berbagai dimensi, bukan hanya fungsionalitas. Kriteria ini mencakup fungsionalitas (kebenaran), kualitas kode (keterbacaan, modularitas), dan efisiensi algoritma (kompleksitas waktu dan ruang). Untuk mencapai evaluasi yang terstruktur dan transparan, LLM memanfaatkan strategi rekayasa prompt (prompt engineering), di mana model diinstruksikan dengan template yang terstruktur. Teknik lanjutan, seperti *Chain of Thought* (CoT) *prompting*, terbukti sangat penting. CoT prompting memecah proses penilaian yang kompleks menjadi langkah-langkah sekuensial. Kerangka kerja seperti StepGrade (Akyash dkk., 2025) memanfaatkan CoT untuk memastikan bahwa kriteria penilaian dianalisis secara terintegrasi dan koheren, meniru penalaran manusia dan menghasilkan umpan balik yang lebih actionable. Model seperti

Claude 3.5 Sonnet, misalnya, telah digunakan untuk memberikan umpan balik berbasis rubrik dengan 22 kriteria terperinci, termasuk Test Case Coverage dan Data Structure Usage (Raj dkk., 2024). Berikut adalah rubrik yang digunakan oleh Raj dkk. (2024).

Tabel II.1 Tabel rubrik kriteria penilaian (Raj dkk., 2024)

Criterion	Pts	Description
Correctness (40 points)		
Test Case Coverage	20	• Passes all provided basic test cases (10 pts) • Passes additional edge cases (10 pts)
Edge Cases Handling	10	• Edge cases are correctly coded (10 pts)
No Syntax Errors	5	• No syntax errors are found (5 pts)
Logical Accuracy	5	• Algorithm is implemented correctly and logically follows the problem requirements (5 pts)
Efficiency (27 points)		
Time Complexity	11	• Meets required time complexity of the problem (6 pts) • Analysis of time complexity shown in comments (5 pts)
Optimal Algorithm	10	• Uses optimal algorithmic method (5 pts) • Considers and implements optimizations (5 pts)
Space Complexity	6	• Meets required space complexity of the problem (3 pts) • Avoids unnecessary data storage and memory allocation (3 pts)
Data Structures (15 points)		
Data Structures	8	• Used suitable data structures for the problem (4 pts) • Explains the choice of data structures in the comments (4 pts)
Efficiency of Data Structures	7	• Data structures are used efficiently (4 pts) • Improves performance with proper usage (3 pts)

Code Readability (10 points)		
Commenting and Documentation	4	• Code is well-commented, explaining logic and important steps (2 pts) • Includes function headers and descriptions (2 pts)
Modular Code	3	• Code is broken down into functions/modules (2 pts) • Functions make sense and have single responsibilities (1 pts)
Naming Conventions	3	• Uses reasonable and consistent variable/function names (2 pts) • Uses correct naming conventions (1 pt)
Testing (8 points)		
Test Case Design	4	• Designs test cases (2 pts) • Test cases cover various scenarios including edge cases (2 pts)
Debugging	2	• Using systematic debugging techniques (1 pt) • Documents debugging process and findings (1 pt)
Error Handling	2	• Implements error handling for invalid inputs (2 pts)

Secara empiris, umpan balik yang dihasilkan LLM telah menunjukkan dampak positif pada pembelajaran dan kinerja siswa. Dalam studi yang menggunakan GPT-3.5, kelompok eksperimen yang menerima petunjuk LLM menunjukkan persentase pengiriman berhasil (successful submissions) yang lebih tinggi pada upaya berturut-turut. Selain itu, perbaikan kode berdasarkan umpan balik berbasis rubrik dari Claude 3.5 Sonnet menghasilkan peningkatan skor rata-rata yang signifikan sebesar 17.5 poin pada tugas pemrograman matematika, dengan peningkatan tertinggi terlihat pada efisiensi algoritma. Meskipun demikian, tantangan tetap ada. LLM cenderung menjadi penilai yang lebih ketat dibandingkan guru manusia, meskipun terdapat korelasi positif yang kuat antara kedua jenis penilaian tersebut (Pearson $R \approx 0.81$) (Jukiewicz, 2023). Selain itu, masalah "halusinasi" LLM (mengidentifikasi kesalahan yang tidak ada) dapat terjadi, tetapi konsistensi penilaian (repeatability) LLM umumnya sangat baik (ICC ≈ 0.95 untuk ChatGPT) dan masalah halusinasi dapat diminimalkan dengan

mengirimkan query ke model setidaknya tujuh kali (Jukiewicz, 2023). Kemunculan model *open-source* seperti Zephyr-7B juga menunjukkan kinerja yang kompetitif dibandingkan model *proprietary* seperti GPT-3.5 dan menawarkan alternatif yang menarik terkait biaya dan masalah privasi data siswa (Koutcheme dkk., 2024).

BAB III

ANALISIS MASALAH DAN RANCANGAN SOLUSI

Fokus utama bab ini adalah mendeskripsikan secara mendalam persoalan yang diangkat dalam tugas akhir dan mengembangkan rancangan solusi. Bab ini akan menguraikan deskripsi umum solusi yang akan diperkuat dengan gambaran konseptual yang jelas sebelum masuk ke tahap perancangan detail.

III.1 Analisis Masalah

Penilaian kode program merupakan aktivitas krusial dalam pendidikan informatika, namun metode yang ada saat ini memiliki kesenjangan (gap) yang signifikan antara kebutuhan ideal dengan realitas operasional. Berdasarkan latar belakang masalah, teridentifikasi persoalan utama sebagai berikut:

1. Penilaian manual memang mampu memberikan umpan balik kualitatif yang mendalam. Namun, metode ini tidak *scalable* dan berujung pada inkonsistensi penilaian dan keterlambatan umpan balik. Oleh karena itu, layanan penilaian otomatis yang mampu bekerja secara masif dan konsisten. Namun, tantangan konsistensi tersebut dipengaruhi oleh halusinasi LLM yang salah satunya disebabkan oleh *Overinflated Self-confidence*. Sehingga, model harus diarahkan agar mampu mengetahui batasan yang diharapkan.
2. Sistem penilaian otomatis seperti Online Judge efektif untuk menilai kebenaran fungsional melalui black-box testing, tetapi tidak mampu menilai kualitas kode secara menyeluruh, seperti keterbacaan, efisiensi, dan gaya penulisan. Akibatnya, kode yang tidak efisien tetap mendapat nilai yang baik. Oleh karena itu, diperlukan layanan penilaian komprehensif dan sesuai dengan penilaian yang diberikan oleh manusia. Tantangan utamanya adalah LLM terkadang menggunakan pengetahuan

bawaannya untuk menghasilkan respon. Hal tersebut harus ditangani dengan mengarahkan LLM agar memiliki kesesuaian dengan penilai.

Dengan demikian, terdapat kebutuhan untuk menjembatani celah antara kecepatan penilaian otomatis dengan kualitas umpan balik penilaian manual.

III.2 Gambaran Umum Solusi

Solusi yang akan dikembangkan dalam penelitian ini adalah layanan penilaian kode program berbasis LLM dengan memanfaatkan teknik Prompt Engineering. Solusi ini dirancang untuk mengatasi permasalahan inkonsistensi penilaian manual dan keterbatasan umpan balik pada penilaian konvensional tanpa memerlukan sumber daya komputasi masif untuk fine-tuning.

Secara garis besar, solusi ini terdiri dari tiga komponen utama: strategi konstruksi prompt (instruksi), arsitektur layanan backend (API), dan antarmuka prototipe untuk kebutuhan pengujian. Gambaran umum solusi dapat dilihat pada lampiran B, Gambar B.1.

III.2.1 Strategi Prompt Engineering

Inti kecerdasan dari layanan ini terletak pada bagaimana instruksi disusun agar LLM dapat bertindak sebagai penilai yang objektif. Sistem tidak akan melatih ulang model, melainkan mengoptimalkan inference melalui teknik berikut:

1. Penerapan Rubrik Terstruktur

Layanan akan menginjeksikan rubrik penilaian spesifik ke dalam prompt. Rubrik ini mengacu pada kriteria kualitas kode seperti kebenaran, efisiensi, dan keterbacaan. Penggunaan rubrik dapat meningkatkan kinerja secara signifikan karena memberikan dasar evaluasi yang jelas.

2. Chain of Thought (CoT)

Instruksi akan dirancang agar model tidak langsung memberikan skor akhir, melainkan melakukan penalaran langkah demi langkah terlebih

dahulu. Teknik ini memecah proses penilaian yang kompleks menjadi analisis sekuensial yang dapat meningkatkan akurasi penilaian.

3. Few-Shot Learning (In-Context Learning)

Untuk menjaga konsistensi format keluaran dan gaya bahasa umpan balik, prompt akan dilengkapi dengan beberapa contoh penilaian yang ideal (pasangan kode input dan hasil penilaian yang diharapkan).

III.2.2 Arsitektur Layanan

Layanan ini akan diimplementasikan sebagai sebuah web service (API) yang bertindak sebagai jembatan antara pengguna dan mesin LLM. Arsitektur sistem terdiri dari modul-modul berikut:

1. Input Handler

Menerima masukan berupa kode program mahasiswa, deskripsi soal, metrik atau rubrik penilaian (apabila tersedia).

2. Prompt Orchestrator

Komponen logika utama yang berfungsi menyusun prompt dinamis. Modul ini menggabungkan kode siswa, konteks soal, rubrik penilaian, dan teknik Chain of Thought menjadi satu kesatuan instruksi yang siap dikirim ke LLM.

3. LLM Engine Interface

Bertugas mengirimkan request ke model LLM melalui API eksternal (misalnya OpenAI GPT-5 atau Google Gemini).

4. Response Parser

Mengolah teks mentah yang dihasilkan oleh LLM, mengekstrak skor dan umpan balik kualitatif, serta memformatnya menjadi struktur data baku (misalnya JSON) agar mudah dikonsumsi oleh aplikasi lain.

Gambaran arsitektur layanan dapat dilihat pada lampiran B, Gambar B.2.

BAB IV

RENCANA PELAKSANAAN

Bab ini bertujuan untuk memberikan gambaran jelas terkait rencana pelaksanaan tugas akhir beserta dengan risiko-risiko yang mungkin dihadapi serta rencana mitigasinya.

IV.1 Jadwal

No	TASK TITLE	FEBRUARI				MARET				APRIL				MEI			
		W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
1	Design																
1.1	Design Class Baseline																
1.2	Design Prompt Baseline																
1.3	Design LLM Usage Plan																
1.4	Design Testing Plan																
2	Implementation																
2.1	Develop Grading Service																
2.1.1	Develop Input Handler																
2.1.2	Develop Prompt Orchestration																
2.1.3	Develop LLM Interface																
2.1.4	Develop Output Parser																
2.2	LLM Integration																
2.3	Develop Simple Interface																
2.4	Interface Integration																
3	Experiment																
3.1	Prompt Strategy Experiment (Iterative)																
3.2	Experiment Analysis & Evaluation																
4	Testing																
4.1	Unit Testing																
4.2	User Testing																
4.3	Load Testing																
4.4	Analysis & Evaluation																
5	Report																
5.1	Report Paper																
5.2	Report Slide																

Gambar IV.1 Jadwal Pelaksanaan Tugas Akhir

Proses pengerjaan tugas akhir dibagi menjadi beberapa tahapan, yaitu tahap *design*, *implementation*, *experiment*, *testing*, dan *report*.

IV.1.1 Design

Pada tahap design akan dilakukan beberapa proses perencanaan seperti perencanaan desain kelas dasar yang akan diimplementasikan, perencanaan design prompt dasar yang akan dijadikan kerangka prompt utama sebelum nantinya diperbaharui seiring dengan berjalannya eksperimentasi, perencanaan penggunaan LLM untuk bisa menggambarkan kebutuhan sumber daya, dan perencanaan skenario testing.

IV.1.2 Implementation

Pada tahap implementation akan dilakukan beberapa proses secara bertahap yang dimulai dari mengembangkan layanan grading terlebih dahulu. Layanan grading tersebut akan dikembangkan secara bertahap per komponennya. Dimulai dari pembuatan handler untuk masukan, pembuatan fungsi orkestrasi prompt, pembuatan jalur komunikasi layanan dengan LLM, dan pembuatan parser hasil LLM agar siap diterima oleh antarmuka. Proses implementation akan dilanjutkan dengan integrasi layanan grading dengan layanan LLM eksternal, pembuatan antarmuka sederhana yang nantinya akan digunakan untuk melakukan pengujian layanan, dan melakukan integrasi antarmuka dengan layanan grading.

IV.1.3 Experiment

Pada tahap experiment akan dilakukan beberapa skenario eksperimen dengan analisis dan juga evaluasinya. Eksperimen yang dilakukan akan terfokus pada eksperimen prompt yang menghasilkan hasil generasi yang terbaik untuk digunakan oleh sistem. Akan dilakukan beberapa skenario eksperimen dengan metode atau teknik yang berbeda-beda. Proses eksperimen akan terus berkembang dan tidak terbatas pada jumlah eksperimen tertentu. Proses eksperimen akan dicukupkan apabila hasil analisis dan evaluasi berhasil memenuhi standar kualitas tertentu.

IV.1.4 Testing

Pada tahap testing akan dilakukan beberapa bentuk pengujian, seperti pengujian dengan menggunakan *Unit Testing*, pengujian pengguna dengan memanfaatkan *User Acceptance Testing*, dan pengujian *robustness* dengan menggunakan *Load Testing*. Tahap ini akan diakhiri dengan analisis dan evaluasi dari hasil pengujian. Akan memungkinkan untuk adanya tahapan tambahan berupa perbaikan apabila hasil analisis dan evaluasi masih kurang dapat memenuhi ukuran ketercapaian tertentu.

IV.1.5 Report

Pada tahap report akan dilakukan dokumentasi serta penyusunan laporan yang akan mencatat seluruh perkembangan tugas akhir beserta pembahasan dan beberapa komponen lainnya.

IV.2 Risiko

Dalam pelaksanaan tugas akhir, ada kemungkinan terjadi beberapa hal yang mungkin dapat menghambat proses pelaksanaan. Beberapa hal yang menjadi risiko dalam pelaksanaan tugas akhir adalah sebagai berikut:

1. Ketersediaan Data

Tujuan dari pengembangan layanan grading ini adalah agar proses penilaian manual dapat digantikan dengan layanan otomatis yang berkualitas. Untuk dapat menjaga kualitas layanan akan diperlukan data latih yang berkualitas juga. Namun, ada kemungkinan bahwa data latih yang dimiliki sudah *outdated* atau berkualitas buruk. Hal tersebut menjadi salah satu risiko yang perlu dimitigasi terlebih dahulu. Ketersediaan data yang minimal harus ditangani dengan menambahkan tahapan pencarian data melalui beberapa cara, seperti pengadaan sesi pengumpulan data baru dengan melibatkan mahasiswa dengan skenario tertentu atau melakukan

penilaian ulang dari hasil jawaban mahasiswa pada ujian atau praktikum tertentu.

2. Ketersediaan Biaya API

Dalam pelaksanaan tugas akhir, diperlukan biaya untuk menggunakan layanan LLM eksternal. Penggunaan yang berlebih dapat menjadi risiko kurangnya biaya untuk menggunakan layanan LLM eksternal. Hal tersebut dapat dimitigasi dengan menggunakan model LLM *open source* dan melakukan proses training di lokal.

3. Ketersediaan Computing Power

Ketersediaan *computing power* dapat menjadi masalah saat melakukan proses eksperimen dalam skala besar. Namun, hal tersebut dapat diatasi dengan melakukan peminjaman *computing power* ataupun melakukan sewa.

4. Ketersediaan User untuk Testing

User yang dibutuhkan untuk testing sendiri adalah spesifik kepada dosen atau penilai. Namun, ada kemungkinan tidak tersedianya waktu untuk dosen atau penilai melakukan pengujian aplikasi. Sehingga, akan dilakukan upaya mitigasi berupa melakukan pengujian mandiri dengan tetap mempertahankan perilaku dosen dan/atau penilai dari berkas-berkas penilaian yang telah dilakukan oleh dosen dan/atau penilai.

5. Kualitas layanan yang masih kurang baik

Ada kemungkinan strategi yang telah diterapkan dalam pengembangan layanan grading masih kurang mampu menghasilkan tingkat kualitas yang tinggi. Apabila hal tersebut terjadi, maka akan dilakukan upaya mitigasi berupa melakukan penyesuaian lebih dalam dengan menggunakan teknik lainnya seperti *fine-tuning* walaupun dengan beberapa pertimbangan seperti biaya komputasi yang kurang sesuai dan kemampuan model untuk tetap *up-to-date*.

DAFTAR PUSTAKA

- Oliveira, D., Bruno, R., Madeiral, F., & Filho, F. (2020). Evaluating Code Readability and Legibility: An Examination of Human-centric Studies. 2020 IEEE International Conference on Software Maintenance and Evolution (ICSME), 348-359. <https://doi.org/10.1109/icsme46990.2020.00041>.
- Messer, M., Brown, N., Kölling, M., & Shi, M. (2024). How Consistent Are Humans When Grading Programming Assignments?. ArXiv, abs/2409.12967. <https://doi.org/10.48550/arxiv.2409.12967>.
- Buyrukoglu, S., Batmaz, F., & Lock, R. (2019). Improving marking efficiency for longer programming solutions based on a semi-automated assessment approach. *Computer Applications in Engineering Education*, 27, 733 - 743. <https://doi.org/10.1002/cae.22094>.
- Insa, D., Pérez, S., Silva, J., & Tamarit, S. (2020). Semiautomatic generation and assessment of Java exercises in engineering education. *Computer Applications in Engineering Education*, 29, 1034 - 1050. <https://doi.org/10.1002/cae.22356>.
- Messer, M., Brown, N., Kölling, M., & Shi, M. (2023). Automated Grading and Feedback Tools for Programming Education: A Systematic Review. *ACM Transactions on Computing Education*, 24, 1 - 43. <https://doi.org/10.1145/3636515>.
- Chen, Z., Chen, J., Wang, W., Zhou, J., Wang, M., Chen, X., Zhou, S., & Wang, J. (2022). Exploring Better Black-Box Test Case Prioritization via Log Analysis. *ACM Transactions on Software Engineering and Methodology*, 32, 1 - 32. <https://doi.org/10.1145/3569932>.
- Loyola-González, O. (2019). Black-Box vs. White-Box: Understanding Their Advantages and Weaknesses From a Practical Point of View. *IEEE Access*, 7, 154096-154113. <https://doi.org/10.1109/access.2019.2949286>.
- Akanksha, V., Amita, K., Sarika, C., (2017). A Comparative Study of Black Box Testing and White Box Testing. *International Journal of Computer Sciences and Engineering*, 5(12), 301-304. <https://doi.org/10.26438/ijcse/v5i12.301304>.
- Min, B., Ross, H., Sulem, E., Veyseh, A., Nguyen, T., Sainz, O., Agirre, E., Heinz, I., & Roth, D. (2021). Recent Advances in Natural Language Processing via Large Pre-trained Language Models: A Survey. *ACM Computing Surveys*, 56, 1 - 40. <https://doi.org/10.1145/3605943>.
- Kim, S., Joo, S. J., Kim, D., Jang, J., Ye, S., Shin, J., & Seo, M. (2023). The CoT Collection: Improving Zero-shot and Few-shot Learning of Language

- Models via Chain-of-Thought Fine-Tuning. ArXiv, abs/2305.14045. <https://doi.org/10.48550/arxiv.2305.14045>.
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2022). Large language models are zero-shot reasoners. ArXiv, abs/2205.11916. <https://doi.org/10.48550/arXiv.2205.11916>.
- Pankiewicz, M., & Baker, R. S. (2023). Large Language Models (GPT) for automating feedback on programming assignments. Proceedings of the 31st International Conference on Computers in Education, Asia-Pacific Society for Computers in Education. <https://doi.org/10.48550/arXiv.2307.00150>
- Koutcheme, C., Dainese, N., Sarsa, S., Hellas, A., Leinonen, J., & Denny, P. (2024). Open Source Language Models Can Provide Feedback: Evaluating LLMs' Ability to Help Students Using GPT-4-As-A-Judge. arXiv, 2405.05253. <https://doi.org/10.48550/arXiv.2405.05253>
- Raj, J., Muppa, A., Dipukumar, A., Nirmal, R., Laddha, A., Kamath, T., Hong, S., Potla, M., & Boicu, M. (2024). Quantitative analysis of rubric-based feedback received from Claude 3.5 Sonnet on mathematical programming problems. 2024 IEEE MIT Undergraduate Research Technology Conference (URTC), Cambridge, MA, USA. <https://doi.org/10.1109/URTC65039.2024.10937532>
- Akyash, M., Azar, K.Z., & Mardani Kamali, H. (2025). StepGrade: Grading Programming Assignments with Context-Aware LLMs. 2025 IEEE Integrated STEM Education Conference (ISEC), 1-6. <https://doi.org/10.48550/arXiv.2503.20851>
- Jukiewicz, M. (2023). The Future of Grading Programming Assignments in Education: The Role of ChatGPT in Automating the Assessment and Feedback Process. Preprint, 10.13140/RG.2.2.22103.85924. <https://doi.org/10.13140/RG.2.2.22103.85924>
- Naveed, H., Khan, A.U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Barnes, N., & Mian, A.S. (2023). A Comprehensive Overview of Large Language Models. ACM Transactions on Intelligent Systems and Technology. <https://doi.org/10.48550/arXiv.2307.06435>
- Cate, M. (2023). Benchmarking Zero-Shot vs. Few-Shot Performance in LLMs. ResearchGate. Preprint, https://www.researchgate.net/publication/388959312_Benchmarking_Zero-Shot_vs_Few-Shot_Performance_in_LLMs
- Zhu, Y., Li, G., Jiang, X., Li, J., Mei, H., Jin, Z., & Dong, Y. (2025). Uncertainty-Guided Chain-of-Thought for Code Generation with LLMs. arXiv, 2503.15341. <https://doi.org/10.48550/arXiv.2503.15341>

- Pathak, A., Gandhi, R., Uttam, V., Ramamoorthy, A., Ghosh, P., Jindal, A. R., Verma, S., Mittal, A., Ased, A., Khatri, C., Nakka, Y., Devansh, Challa, J. S., & Kumar, D. (2025). Rubric Is All You Need: Enhancing LLM-based Code Evaluation With Question-Specific Rubrics. arXiv, 2503.23989. <https://doi.org/10.48550/arXiv.2503.23989>
- Zhang, Y., Li, Y., Cui, L., Cai, D., Liu, L., Fu, T., Huang, X., Zhao, E., Zhang, Y., Xu, C., Chen, Y., Wang, L., Luu, A. T., Bi, W., Shi, F., & Shi, S. (2025). Siren's song in the AI ocean: A survey on hallucination in large language models (Version 3). arXiv, 2309.01219. <https://doi.org/10.48550/arXiv.2309.01219>

Lampiran A. Analisis Solusi

Alternatif 1: Pemanfaatan Large Language Model (LLM) tanpa Penyesuaian

Alternatif pertama merepresentasikan pendekatan paling langsung dalam menggunakan LLM yang telah dilatih sebelumnya (pre-trained models) dalam mode inferensi zero-shot. Dalam konfigurasi ini, sistem penilaian tidak melakukan penyesuaian apapun baik dengan teknik *prompting engineering*, dengan melakukan pelatihan ulang (fine-tuning) terhadap parameter model, maupun dengan menyediakan contoh-contoh penilaian spesifik (few-shot examples) dalam konteks input. Sebaliknya, sistem bergantung sepenuhnya pada pengetahuan dasar yang telah dimiliki oleh model.

Secara operasional, sistem menyusun sebuah instruksi tunggal (prompt) yang berisi kode siswa yang akan dinilai beserta deskripsi soal. Model kemudian diminta untuk melakukan evaluasi kualitatif dan kuantitatif berdasarkan pemahamannya sendiri tentang sintaksis bahasa pemrograman, logika algoritma, dan konvensi pengkodean umum. Pendekatan ini menggunakan kemampuan generalisasi model seperti pengetahuan sintaksis dan semantik dari data pelatihan untuk menyelesaikan tugas penilaian spesifik yang belum pernah dilihat sebelumnya.

Penggunaan LLM tanpa penyesuaian (Zero-shot) menggunakan konsep "kemampuan yang muncul" (emergent abilities) pada model skala besar. Hal ini dapat menghemat sumber daya karena tidak perlu dilakukan eksperimen lebih lanjut dan juga token yang digunakan dalam tiap promptnya akan lebih sedikit dibandingkan metode *prompt engineering*. Namun, meskipun metode ini menunjukkan kinerja baseline yang kuat, literatur mengidentifikasi keterbatasan kritis dalam mode zero-shot. Ketiadaan konteks eksternal atau panduan spesifik membuat model rentan terhadap bias internal yang diperoleh selama *pre-training*. Studi perbandingan antara kinerja zero-shot dan few-shot menunjukkan bahwa meskipun zero-shot unggul dalam tugas klasifikasi teks sederhana dan analisis sentimen, performanya sering kali menurun secara signifikan pada tugas penalaran kompleks dan terstruktur seperti penilaian kode tingkat lanjut. Model

zero-shot cenderung memberikan penilaian yang "halus" atau terlalu umum, gagal menangkap nuansa spesifik dari rubrik penilaian yang mungkin berbeda dari perilaku yang dipelajari model. Misalnya, model mungkin mengabaikan persyaratan gaya pengkodean spesifik universitas jika tidak diinstruksikan secara eksplisit atau diberikan contoh. Hal ini menjadikan metode *zero-shot* ini adalah metode yang harus dihindari untuk melakukan tugas dengan kompleksitas yang tinggi.

Alternatif 2: Pemanfaatan Large Language Model (LLM) dengan Prompt Engineering

Alternatif kedua menggunakan LLM untuk mengevaluasi kode dengan menambahkan teknik penyesuaian *Prompt Engineering*. Pendekatan ini difokuskan pada pemanfaatan teknik *Few-shot* dan Chain-of-Thought untuk mengarahkan model bertindak sebagai penilai. Teknik ini juga dikonfigurasi agar menggunakan informasi tambahan seperti metrik penilaian ataupun rubrik penilaian yang telah disesuaikan dengan kebutuhan penilai manusia.

Secara operasional, sistem akan menyusun instruksi (prompt) yang berisi deskripsi soal atau permasalahan yang akan diselesaikan oleh kode, kode siswa yang akan dinilai, metrik penilaian kode (baik metrik umum ataupun metrik yang lebih spesifik), dan juga rubrik penilaian soal yang diberikan oleh penilai manusia. Model kemudian diminta untuk melakukan evaluasi kualitatif dan kuantitatif berdasarkan instruksi dan/atau contoh yang diberikan dan juga pengetahuan dasar yang dimiliki model seperti pengetahuan terkait bahasa pemrograman, logika algoritma, dan konvensi pengkodean umum. Pendekatan ini menggunakan kemampuan generalisasi model untuk menangkap instruksi dan juga beberapa contoh yang diberikan untuk kemudian dijadikan pengetahuan tambahan sebagai panduan dan pelengkap pengetahuan dasar yang telah dimiliki model.

Beberapa justifikasi metode ini adalah LLM telah dilatih pada dataset kode masif sehingga mampu memahami struktur logis dan memberikan penjelasan menyerupai manusia. LLM juga dapat diinstruksikan untuk menilai aspek kualitatif seperti "Keterbacaan" atau "Efisiensi" menggunakan rubrik terstruktur (seperti rubrik 22 kriteria oleh Raj dkk.). Fokus dari metode ini yang hanya ditujukan kepada *Prompt Engineering* dan penggunaan API memungkinkan pengembangan layanan yang cepat tanpa perlu melakukan *pre-training* model dari awal maupun *fine-tuning* model yang memakan sumber daya besar.

Dari segi performa, dekomposisi masalah dalam teknik *Chain-of-Thought* secara signifikan meningkatkan kinerja model dalam pemecahan masalah pemrograman (Zhu dkk., 2025). Penggunaan rubrik penilaian juga dapat meningkatkan performa layanan dalam melakukan penilaian.

“Providing a rubric gives the LLM grader an anchor around which to evaluate or grade the code, thereby increasing performance significantly when a rubric is provided” (Pathak dkk., 2025).

Oleh karena itu, metode ini sangat cocok untuk digunakan sebagai dasar dari layanan penilaian kode program yang komprehensif.

Alternatif 3: Pemanfaatan Large Language Model (LLM) dengan Fine-Tuning

Alternatif ketiga menggunakan LLM untuk mengevaluasi kode dengan melakukan penyesuaian bobot parameter LLM tersebut terlebih dahulu. Pendekatan ini akan melakukan penyesuaian bobot parameter LLM untuk mengarahkan model bertindak sebagai penilai dengan menggunakan pengetahuan-pengetahuan terkait penilaian saat ini. Teknik ini juga dapat dikonfigurasi agar menggunakan informasi tambahan seperti metrik penilaian ataupun rubrik penilaian yang telah disesuaikan dengan kebutuhan penilai manusia.

Secara operasional, model akan dilatih ulang dengan informasi spesifik seperti pengetahuan pemrograman dan penilaian. Model kemudian diminta untuk

melakukan evaluasi kualitatif dan kuantitatif berdasarkan instruksi tunggal dengan memanfaatkan pengetahuan dasar yang telah disesuaikan. Pendekatan ini menggunakan kemampuan generalisasi model untuk menangkap instruksi dan juga permasalahan yang akan diselesaikan untuk kemudian dijadikan pengetahuan tambahan sebagai panduan dan pelengkap pengetahuan yang telah disesuaikan.

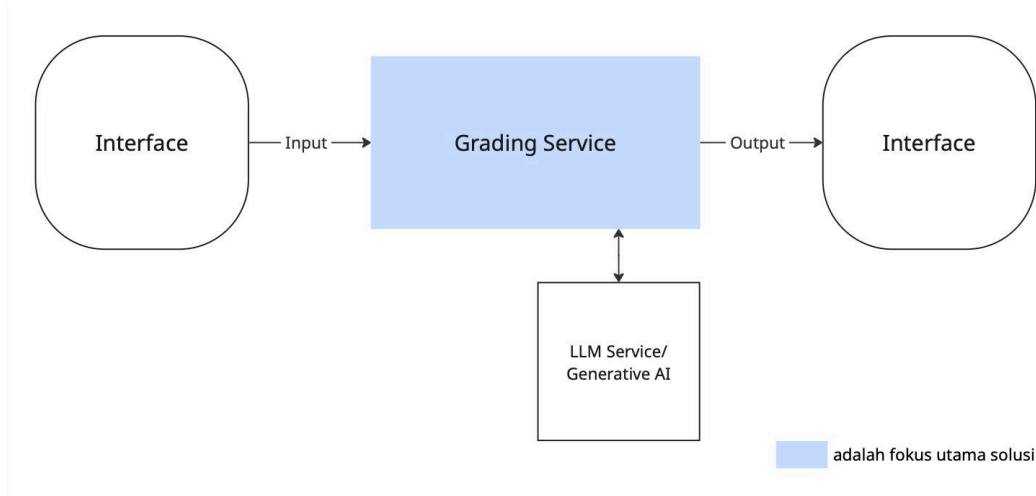
Beberapa justifikasi metode ini adalah kemampuan model untuk mengurangi penggunaan token dalam setiap request karena pengetahuan tambahan yang diperlukan akan disesuaikan melalui pelatihan model bukan melalui *prompt*. Namun hal tersebut justru menjadikan metode ini tidak *cost-efficient*. Metode ini memerlukan sumber daya yang sangat besar untuk melakukan pelatihan ulang. Justifikasi lainnya adalah model yang telah dilatih menggunakan data atau pengetahuan saat ini dapat berakibat pada ketertinggalan pengetahuan yang dimiliki oleh model pada masa yang akan datang yang membuat model perlu dilakukan penyesuaian pengetahuan ulang.

Berdasarkan beberapa analisis tersebut, Alternatif 2 dipilih karena menawarkan keseimbangan terbaik antara otomatisasi, kualitas umpan balik, dan kelayakan implementasi.

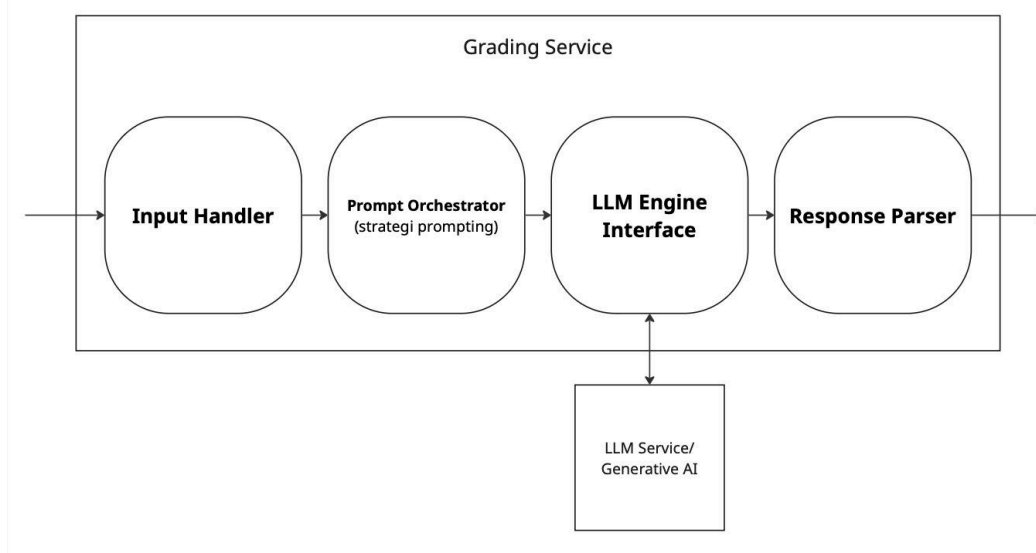
Meskipun fokus utama penelitian ini adalah pada layanan backend, sebuah antarmuka pengguna sederhana berbasis web akan dibangun sebagai kakas pendukung. Antarmuka ini tidak menjadi bagian inti dari solusi layanan, namun keberadaannya krusial untuk:

1. Memfasilitasi pengujian validasi oleh pengguna (dosen/penilai) guna mendapatkan umpan balik terkait kesesuaian penilaian.
2. Menampilkan umpan balik kualitatif (saran perbaikan) secara terstruktur kepada pengguna.

Lampiran B. Gambaran Solusi



Gambar B.1 Gambaran Umum Solusi



Gambar B.2 Gambar Arsitektur Layanan Penilaian